

My office hours are in 779 Soda:

- Mondays, 3:10 - 3:58 PM [ending a couple minutes early to get to lecture]
- Thursdays, 2:00 - 3:00 PM

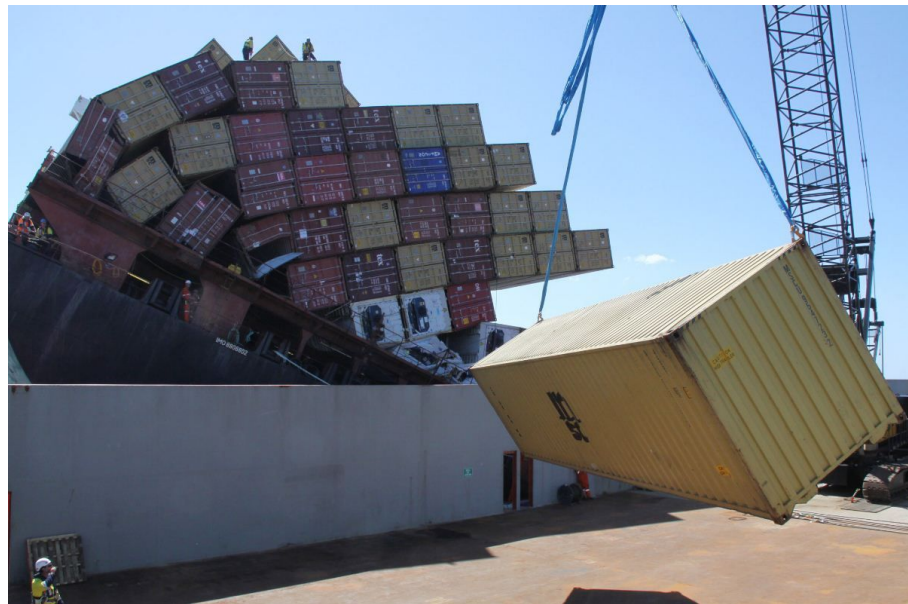
Come talk about anything.

- Top priority: Conceptual / theoretical material related to 61B.
- Bottom priority: Specific homework or project questions. Debugging.
- Middle priority: Anything else (life questions, grad school, chatting about recent news, showing off your cool tattoo of a giant squid, whatever).

Lecture 5 (Lists 3)

DLLists and Basic ALists

CS61B, Fall 2025 @ UC Berkeley



Summary of SLLists So Far

Lecture 5, CS61B, Fall 2025

SLLists:

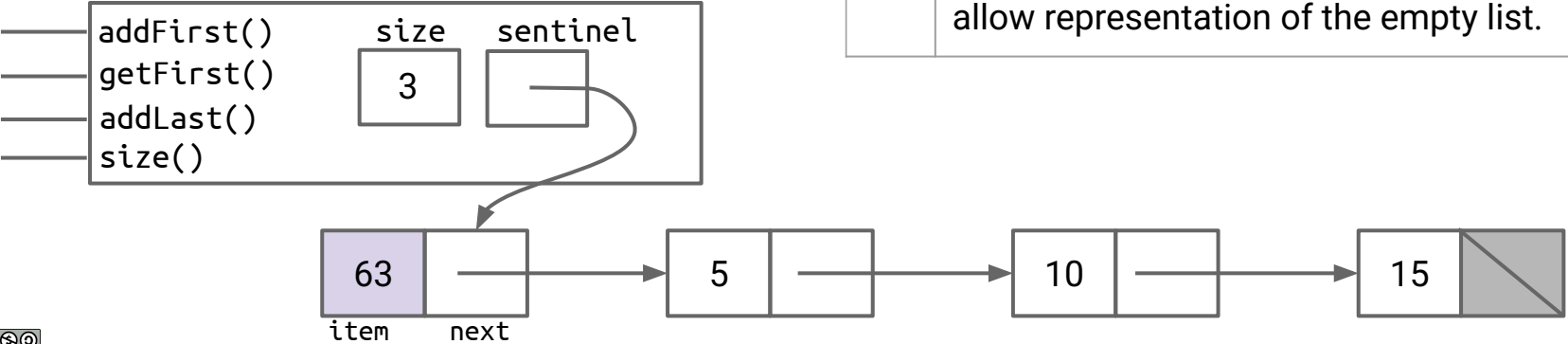
- **Summary of SLLists So Far**
- Why a Last Pointer Isn't Enough
- Doubly Linked Lists
- Generic Lists
- Linked Lists Are Bad at Get

ALists:

- Array Review
- Basic ArrayList Implementation
- removeLast and the Allegory of the Cave
- removeLast Implementation

Summary of Last Time (From IntList to SLList)

Methods	Non-Obvious Improvements	
addFirst(int x)	#1	Rebranding: <code>IntList</code> → <code>IntNode</code>
getFirst()	#2	Bureaucracy: <code>SLList</code>
addLast(int x)	#3	Access Control: <code>public</code> → <code>private</code>
size()	#4	Nested Class: Bringing <code>IntNode</code> into <code>SLList</code>
	#5	Caching: Saving size as an int.
	#6	Generalizing: Adding a sentinel node to allow representation of the empty list.

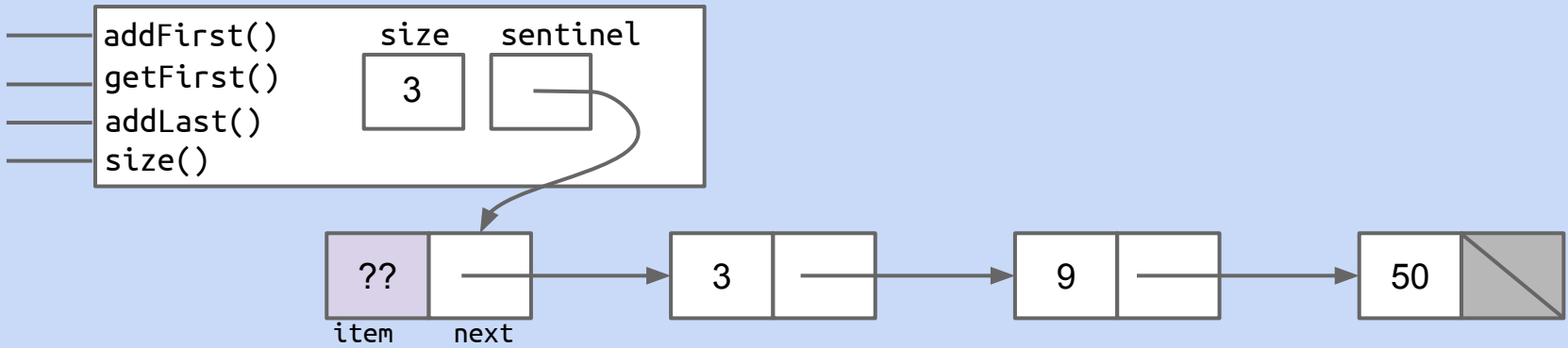


Inserting at the back of an SLList is much slower than the front.

```
public void addFirst(int x) {  
    sentinel.next = new IntNode(x, sentinel.next);  
}
```

```
public void addLast(int x) {  
    size += 1;  
  
    IntNode p = sentinel;  
    while (p.next != null) {  
        p = p.next;  
    }  
  
    p.next = new IntNode(x, null);  
}
```

How could we modify our list data structure so that addLast is also fast?



Why a Last Pointer Isn't Enough

Lecture 5, CS61B, Fall 2025

SLLists:

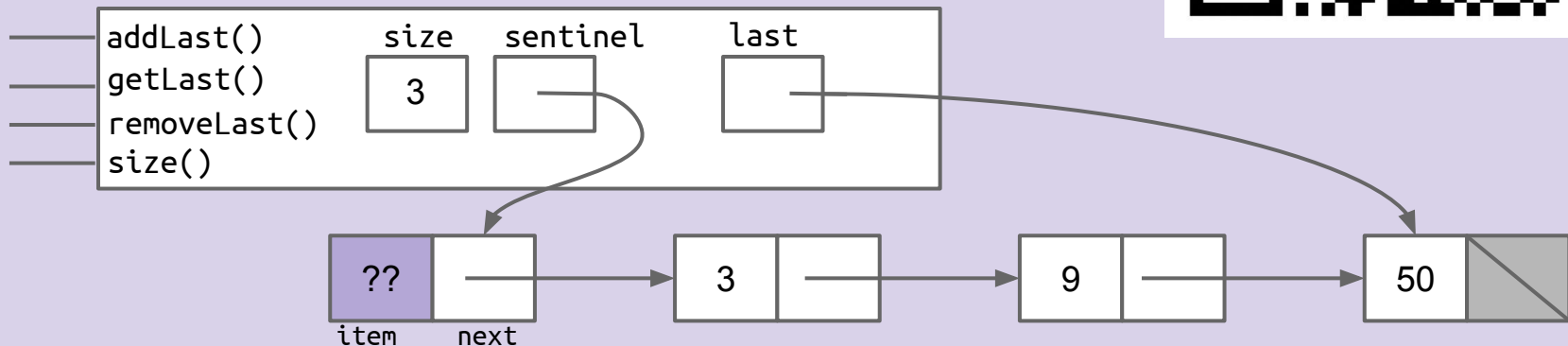
- Summary of SLLists So Far
- **Why a Last Pointer Isn't Enough**
- Doubly Linked Lists
- Generic Lists
- Linked Lists Are Bad at Get

ALists:

- Array Review
- removeLast and the Allegory of the Cave
- removeLast Implementation

Suppose we want to support **addLast**, **getLast**, and **removeLast**, will having a last pointer make these methods fast, even on long lists?

- A. Yes
- B. No, addLast would be slow for long lists.
- C. No, getLast would be slow for long lists.
- D. No, removeLast would be slow for long lists.



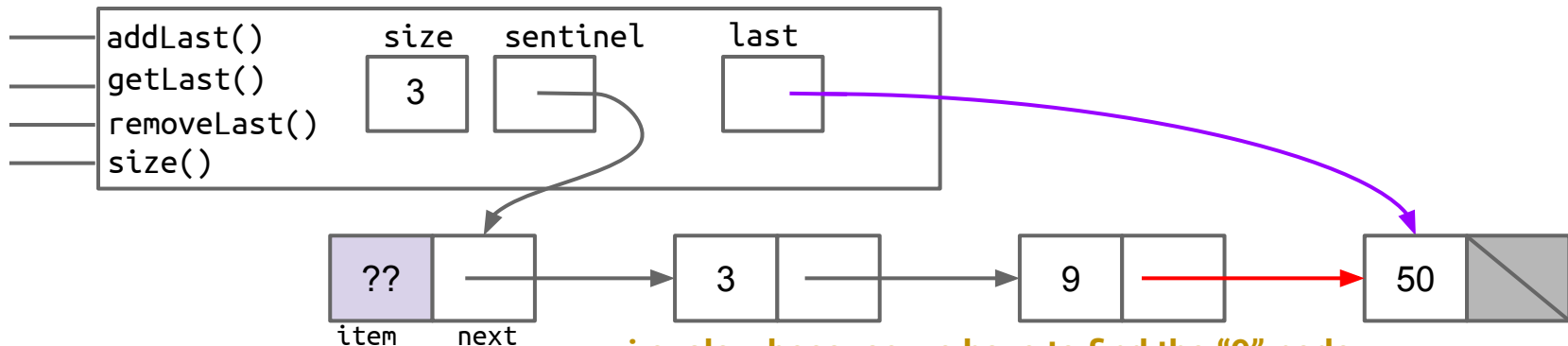
.last Is Not Enough

Suppose we want to support `addLast`, `getLast`, and `removeLast`, will having a last pointer make these methods fast, even on long lists?

- **No, `removeLast` would be slow for long lists.**

`removeLast` requires two actions:

- Setting **9's next variable** to null.
- Setting **last** equal to 9's memory location.
 - Have to search through list to find the 9 node (second to last).



i.e. slow because we have to find the "9" node.

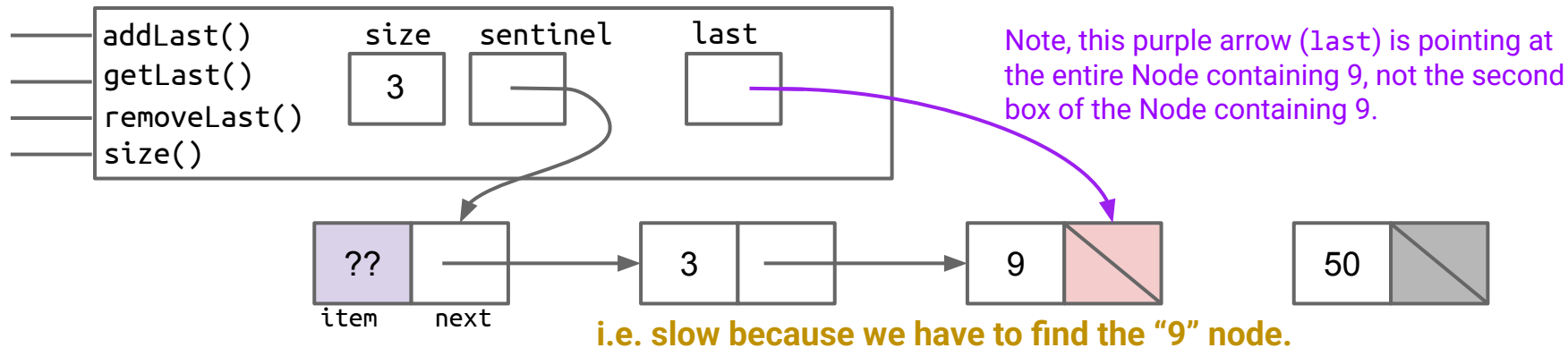
.last Is Not Enough

Suppose we want to support `addLast`, `getLast`, and `removeLast`, will having a `last` pointer make these methods fast, even on long lists?

- **No, `removeLast` would be slow for long lists.**

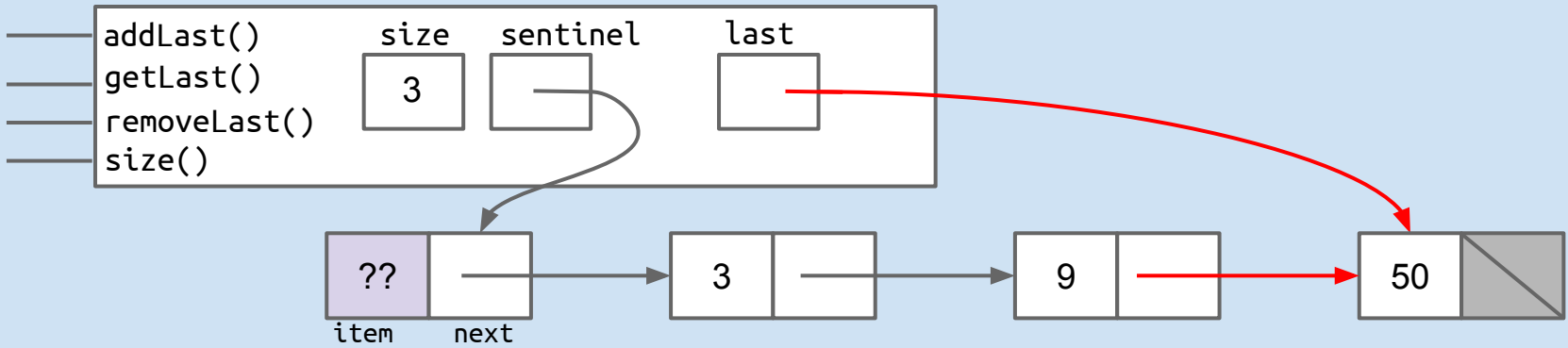
`removeLast` requires two actions:

- Setting **9's next variable** to null.
- Setting **last** equal to 9's memory location.
 - Have to search through list to find the 9 node (second to last).



Improvement #7: .last and ??? Goal: Fast operations on last.

We added .last, which will speed up addLast and removeLast. What other changes might we make so that removeLast is also fast?



Doubly Linked Lists

Lecture 5, CS61B, Fall 2025

SLLists:

- Summary of SLLists So Far
- Why a Last Pointer Isn't Enough
- **Doubly Linked Lists**
- Generic Lists
- Linked Lists Are Bad at Get

ALists:

- Array Review
- Basic ArrayList Implementation
- removeLast and the Allegory of the Cave
- removeLast Implementation

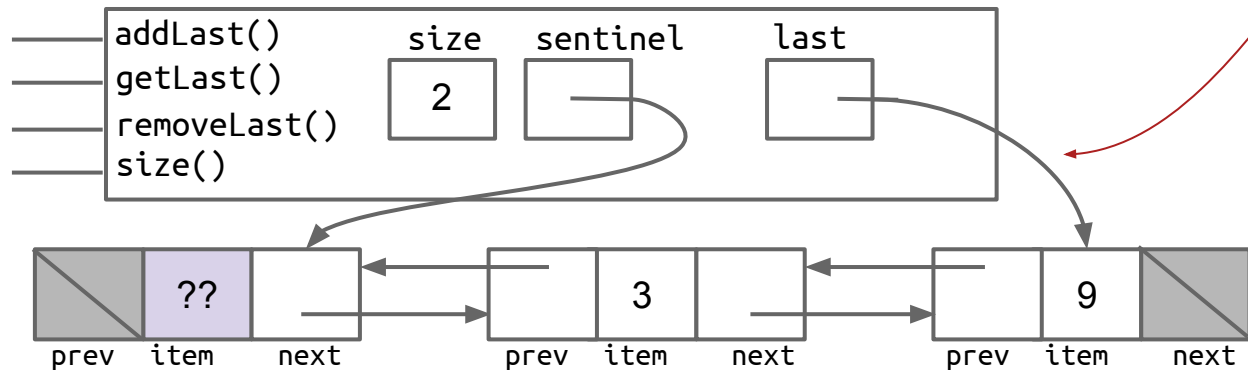
Improvement #7: .last and .prev

We added .last. What other changes might we make so that removeLast is also fast?

- Add backwards links from every node.
- This yields a “**doubly linked list**” or **DLList**, as opposed to our earlier “**singly linked list**” or **SLList**.

Note: Arrows point at entire nodes, not fields!

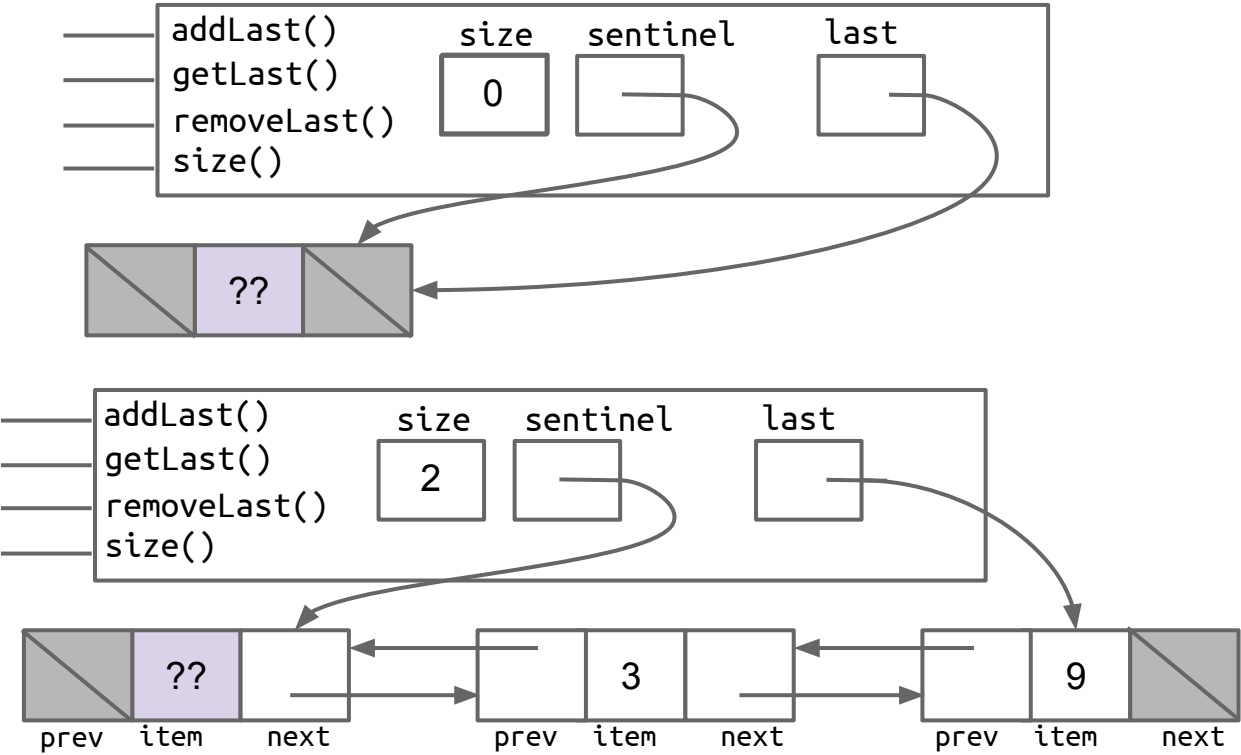
Example: last holds the address of the last node, not the item field of the sentinel node.



Doubly Linked Lists (Naive)

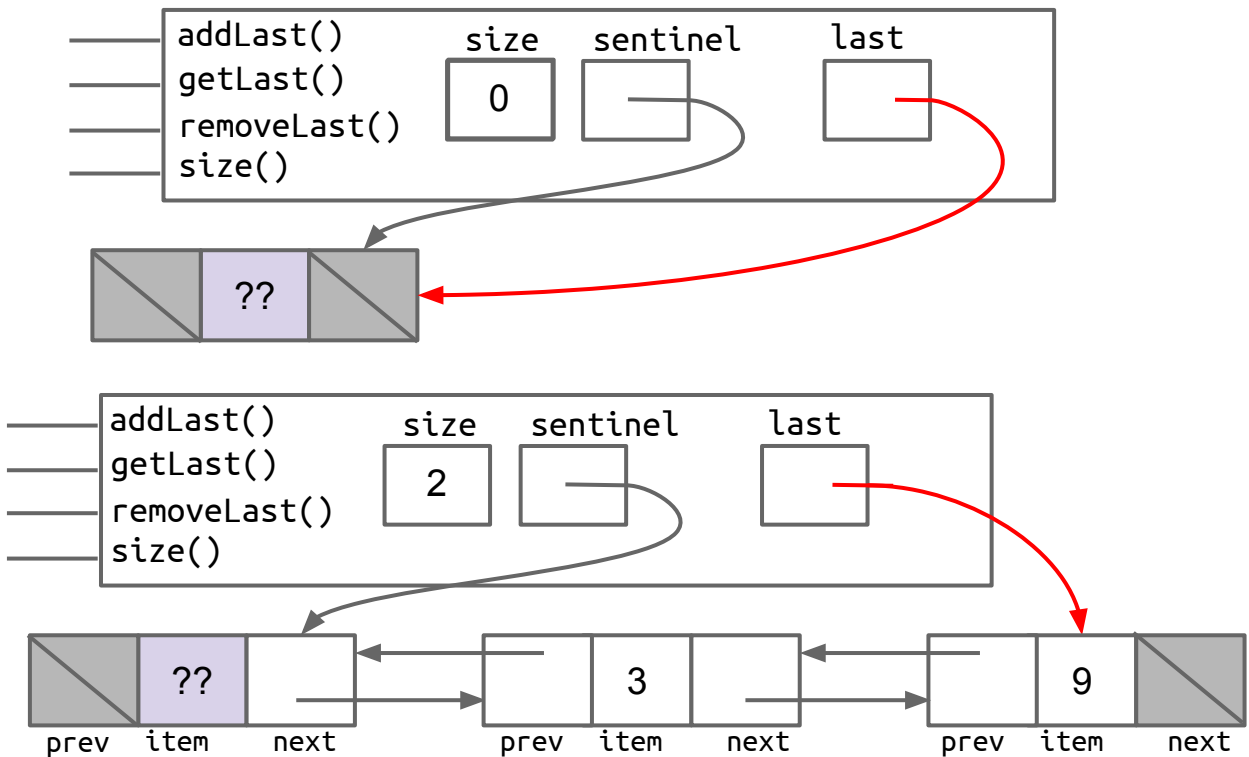
Reverse pointers allow all operations (add, get, remove) to be fast.

- We call such a list a “doubly linked list” or **DLList**.



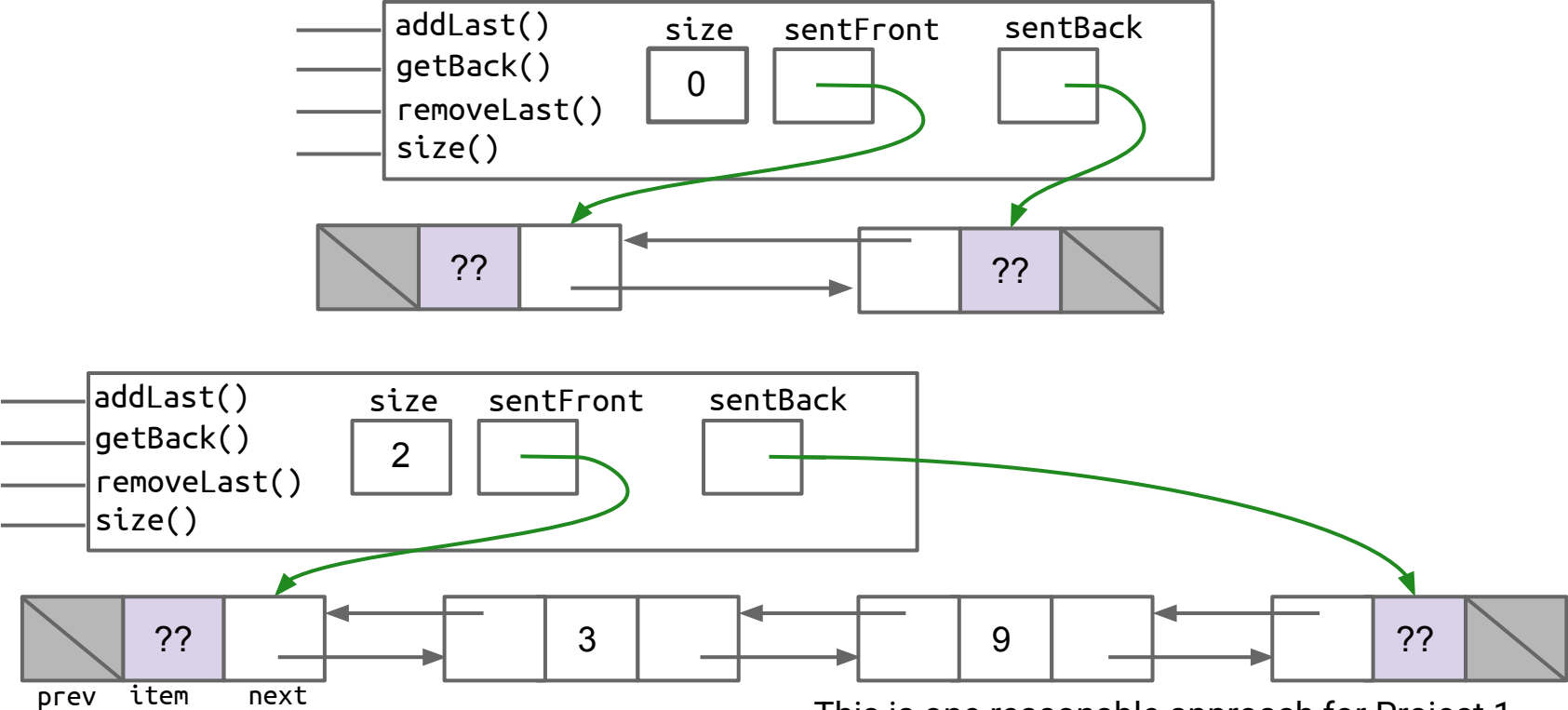
Doubly Linked Lists (Naive)

Non-obvious fact: This approach has an annoying special case: last sometimes points at the sentinel, and sometimes points at a 'real' node.



Doubly Linked Lists (Double Sentinel)

One solution: Have two sentinels.

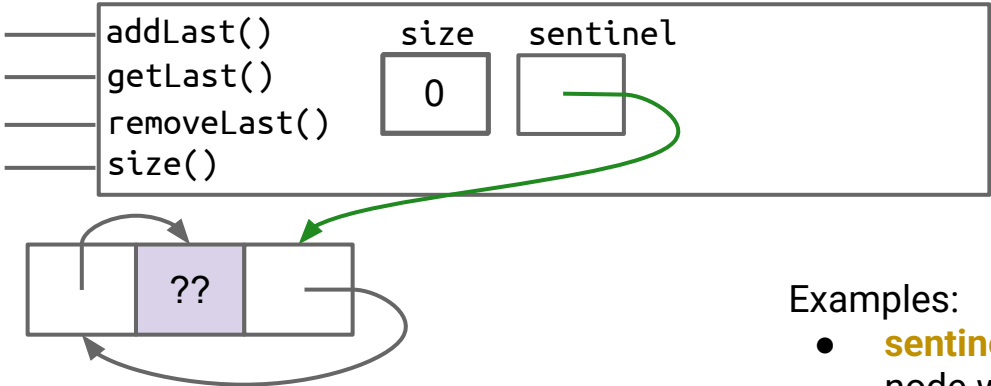


This is one reasonable approach for Project 1.

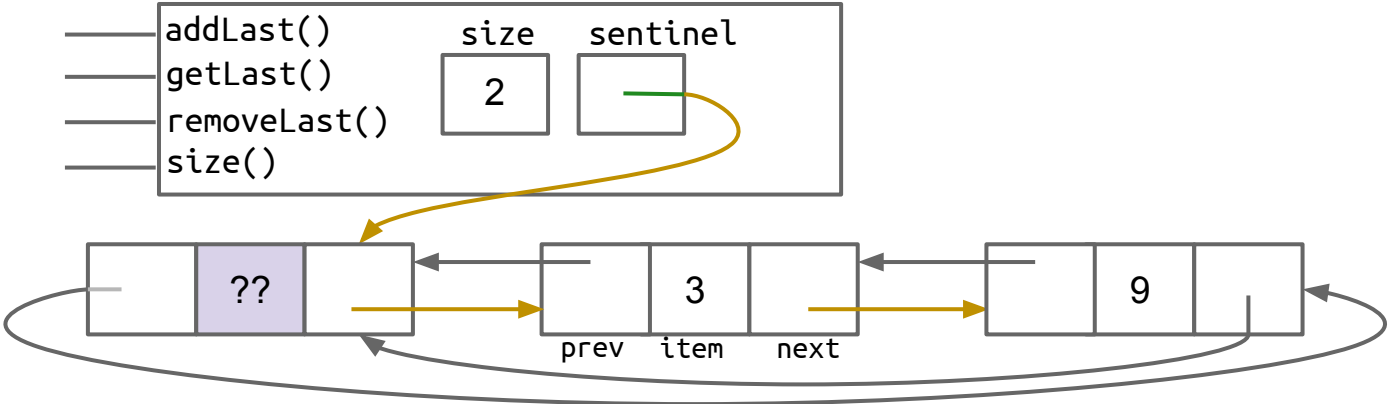
Doubly Linked Lists (Circular Sentinel)

Even better topology (IMO):

Note: arrows are pointing at entire nodes, not specific fields of nodes.



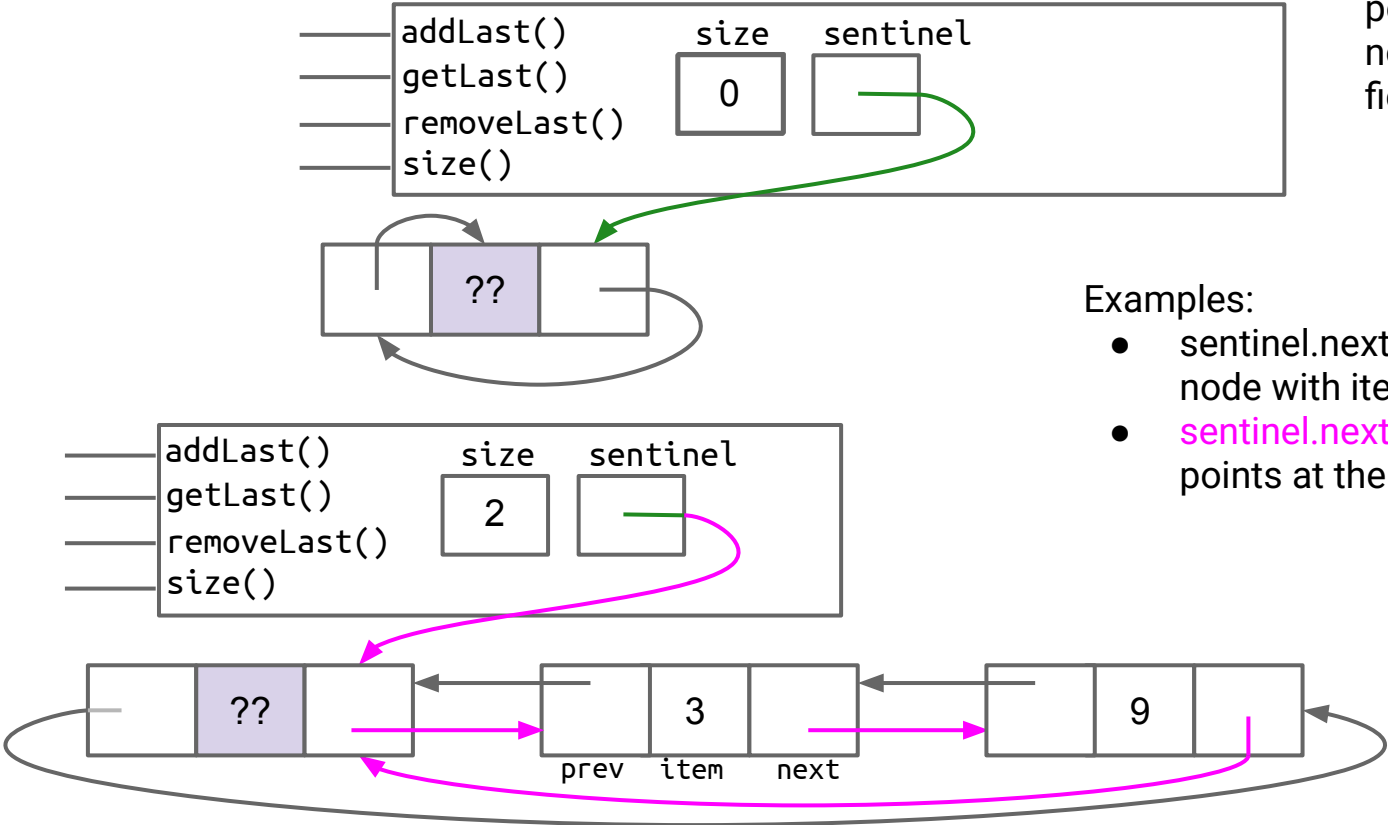
- Examples:
- **sentinel.next.next** is the node with item=9.



Doubly Linked Lists (Circular Sentinel)

Even better topology (IMO):

Note: arrows are pointing at entire nodes, not specific fields of nodes.



Improvement #8: Fancier Sentinel Node(s)

While fast, adding `.last` and `.prev` introduces lots of special cases.

To avoid these, either:

- Add an additional `sentBack` sentinel at the end of the list.
- Make your linked list circular (required for mini-project 1, with a single sentinel in the middle).

DLList Summary

Methods	Non-Obvious Improvements	
<code>addFirst(int x)</code>	#1	Rebranding: <code>IntList</code> → <code>IntNode</code>
<code>getFirst()</code>	#2	Bureaucracy: <code>SLList</code>
<code>size()</code>	#3	Access Control: <code>public</code> → <code>private</code>
<code>addLast(int x)</code>	#4	Nested Class: Bringing <code>IntNode</code> into <code>SLList</code>
<code>removeLast()</code>	#5	Caching: Saving size as an <code>int</code> .
	#6	Generalizing: Adding a <code>sentinel</code> node to allow representation of the empty list.
	#7	Looking back: <code>.last</code> and <code>.prev</code> allow fast <code>removeLast</code>
	#8	Sentinel upgrade: Avoiding special cases with <code>sentBack</code> or circular list.

Still many steps before we have an industrial strength data structure. Will discuss over next two weeks.

Generic Lists

Lecture 5, CS61B, Fall 2025

SLLists:

- Summary of SLLists So Far
- Why a Last Pointer Isn't Enough
- Doubly Linked Lists
- **Generic Lists**
- Linked Lists Are Bad at Get

ALists:

- Array Review
- Basic ArrayList Implementation
- removeLast and the Allegory of the Cave
- removeLast Implementation

Integer Only Lists

One issue with our list classes: They only support integers.

```
public class SLList {  
    private IntNode sentinel;  
    private int size;
```

```
    public class IntNode {  
        public int item;  
        public IntNode next;  
        ...  
    }  
    ...  
}
```

```
SLList s1 = new SLList(5);  
s1.addFirst(10);
```

Works fine!

```
SLList s2 = new SLList("hi");  
s2.addFirst("apple");
```

```
SLListLauncher.java:6: error:  
incompatible types: String cannot  
be converted to int
```

```
SLList s2 = new SLList("hi");
```

Coding Demo: Generic Lists

SLList.java

```
public class SLList {  
    private IntNode sentinel;  
    private int size;  
  
    private class IntNode {  
        public int item;  
        public IntNode next;  
  
        public IntNode(int i, IntNode n) {  
            item = i;  
            next = n;  
        }  
    }  
}
```

In this demo, we'll modify our SLList to support lists of any data type, not just lists of integers.

Coding Demo: Generic Lists

SLList.java

```
public class SLList<LochNess> {  
    private IntNode sentinel;  
    private int size;  
  
    private class IntNode {  
        public int item;  
        public IntNode next;  
  
        public IntNode(int i, IntNode n) {  
            item = i;  
            next = n;  
        }  
    }  
}
```

← A placeholder name, which will get replaced by the true data type each time a new SLList is created.

Coding Demo: Generic Lists

SLList.java

```
public class SLList<LochNess> {  
    private IntNode sentinel;  
    private int size;  
  
    private class IntNode {  
        public LochNess item;  
        public IntNode next;  
  
        public IntNode(LochNess i, IntNode n) {  
            item = i;  
            next = n;  
        }  
    }  
}
```

Items are no longer integers, but the LochNess placeholder data type.

Coding Demo: Generic Lists

SLList.java

```
public class SLList<LochNess> {  
    private StuffNode sentinel;  
    private int size;  
  
    private class StuffNode {  
        public LochNess item;  
        public StuffNode next;  
  
        public StuffNode(LochNess i, StuffNode n) {  
            item = i;  
            next = n;  
        }  
    }  
}
```

Renaming IntNode
to StuffNode to be
more descriptive.

SLList.java

```
public class SLList<LochNess> {  
    private StuffNode sentinel;  
    private int size;  
  
    public SLList(LochNess x) {  
        sentinel = new StuffNode(null, null);  
        sentinel.next = new StuffNode(x, null);  
        size = 1;  
    }  
  
    public SLList() {  
        sentinel = new StuffNode(null, null);  
        size = 0;  
    }  
}
```

Replaced int x with
LochNess x, the
placeholder data
type.

Coding Demo: Generic Lists

SLList.java

```
public class SLList<LochNess> {  
    private StuffNode sentinel;  
    private int size;  
  
    public void addFirst(LochNess x) {  
        sentinel.next = new StuffNode(x, sentinel.next);  
        size += 1;  
    }  
  
    public LochNess getFirst() {  
        return sentinel.next.item;  
    }  
}
```

Replaced int x with LochNess x, the placeholder data type.

Return type is LochNess, not int.

Coding Demo: Generic Lists

SLList.java

```
public class SLList<LochNess> {  
    private StuffNode sentinel;  
    private int size;  
  
    public void addLast(LochNess x) {  
        size += 1;  
        StuffNode p = sentinel;  
  
        /** Move p until it reaches the end of the list. */  
        while (p.next != null) {  
            p = p.next;  
        }  
        p.next = new StuffNode(x, null);  
    }  
}
```

Replaced int x with
LochNess x, the
placeholder data
type.

Java allows us to defer type selection until declaration.

```
public class SLList<BleepBlorp> {  
    private IntNode sentinel;  
    private int size;  
  
    public class IntNode {  
        public BleepBlorp item;  
        public IntNode next;  
        ...  
    }  
  
    ...  
}
```

```
SLList<Integer> s1 = new SLList<>(5);  
s1.addFirst(10);  
  
SLList<String> s2 = new SLList<>("hi");  
s2.addFirst("apple");
```

We won't get into the various complexities around generics in 61B. Here are the rules of thumb you'll need for our course:

- In the .java file **implementing** your data structure, specify your “generic type” **only once** at the very top of the file.
- In .java files that **use** your data structure, specify desired type **once**:
 - Write out desired type during **declaration**.
 - Use the empty diamond operator `<>` during **instantiation**.
- Make the nested class non-static (there's a way around this, ask at OH).
- When declaring or instantiating your data structure, use the reference type.
 - `int`: `Integer`
 - `double`: `Double`
 - `char`: `Character`
 - `boolean`: `Boolean`
 - `long`: `Long`
 - etc.

```
DLList<Double> s1 = new DLList<>(5.3);  
  
double x = 9.3 + 15.2;  
s1.addFirst(x);
```

Linked Lists Are Bad at Get

Lecture 5, CS61B, Fall 2025

SLLists:

- Summary of SLLists So Far
- Why a Last Pointer Isn't Enough
- Doubly Linked Lists
- Generic Lists
- **Linked Lists Are Bad at Get**

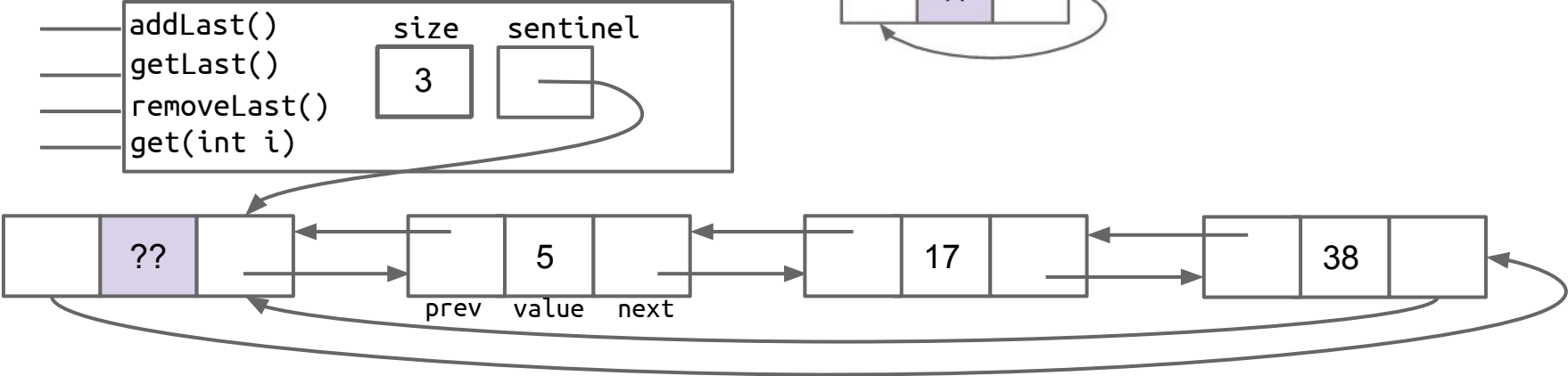
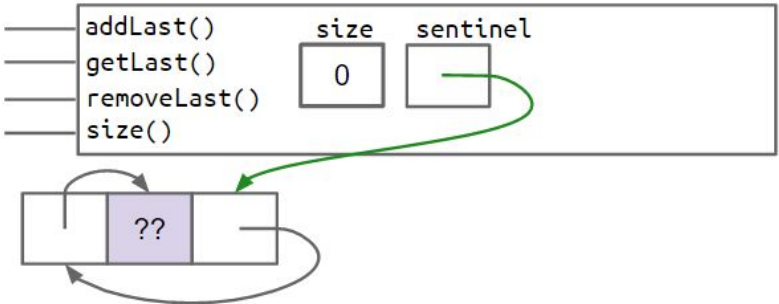
ALists:

- Array Review
- Basic ArrayList Implementation
- removeLast and the Allegory of the Cave
- removeLast Implementation

Doubly Linked Lists

Good news: We've designed a linked list that can handle the following operations efficiently.

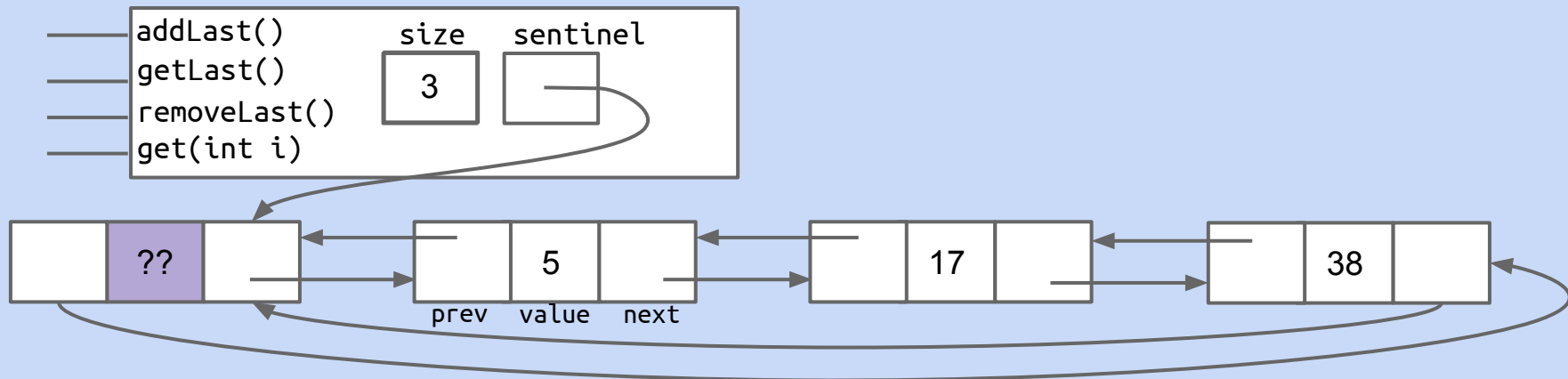
- addFirst, addLast
- getFirst, getLast
- removeFirst, removeLast
- You will build this in project 1.



Bad news: Can't efficiently implement `get(int i)`.

Suppose we added `get(int i)`, which returns the *i*th item from the list.

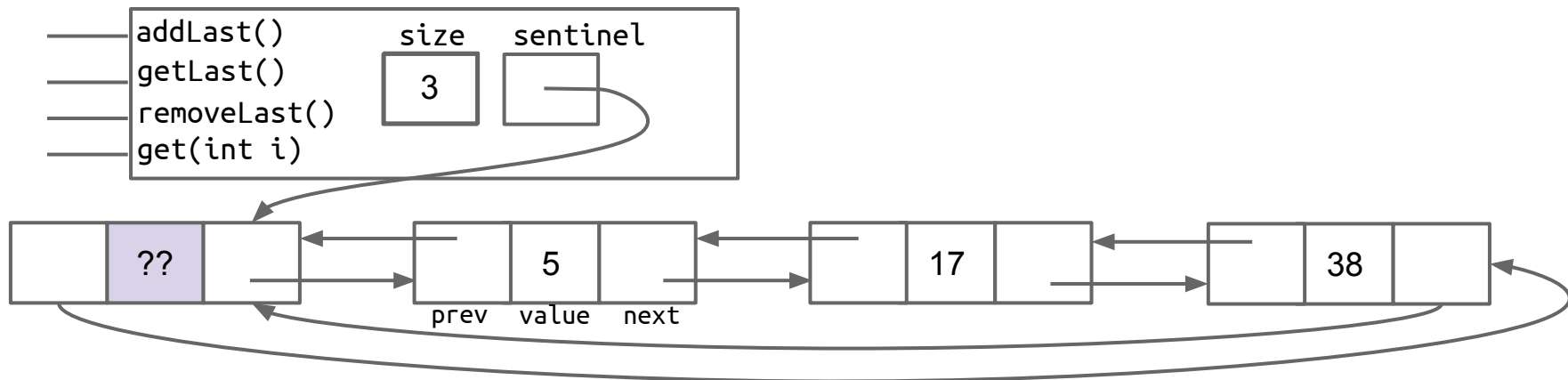
Why would `get` be slow for long lists compared to `getLast()`? For what inputs?



Suppose we added `get(int i)`, which returns the *i*th item from the list.

Why would `get` be slow for long lists compared to `getLast()`? For what inputs?

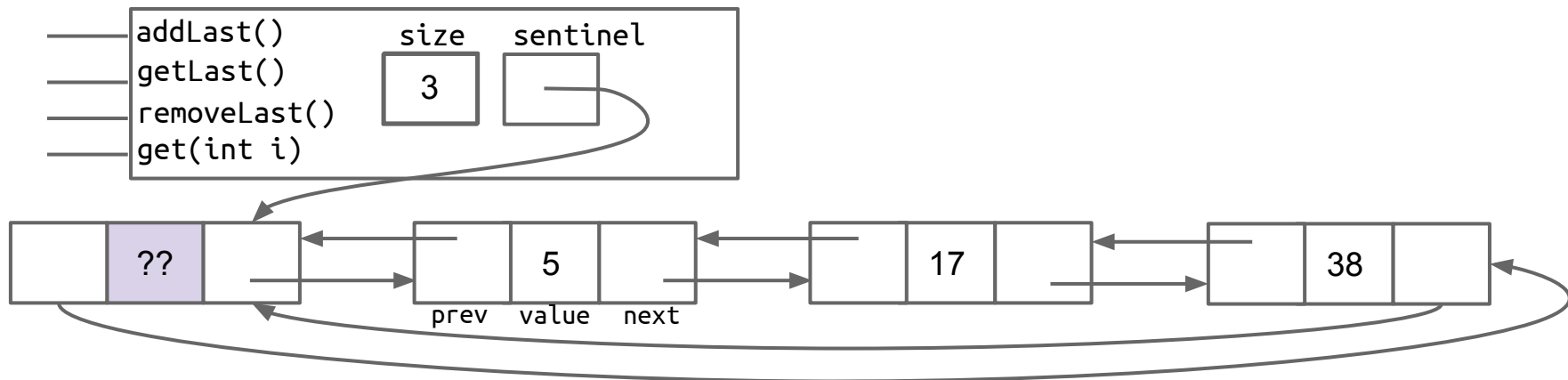
- Have to scan to desired position. Slow for any *i* not near the sentinel node.
- How do we fix this?



Suppose we added `get(int i)`, which returns the *i*th item from the list.

Why would `get` be slow for long lists compared to `getLast()`? For what inputs?

- Have to scan to desired position. Slow for any *i* not near the sentinel node.
- Will discuss (much later) sophisticated changes that can speed things up.
- For today: We'll take a different tack: Using an array instead (no links!).



At this point we've covered everything you need to know to complete Mini-Project 1.

- On mini-project 1, you'll implement a doubly linked list.
 - Builds very naturally on the Stack you implemented for homework.
- The project includes a bit of syntax we won't formally cover in lecture, specifically the idea of a "class implementing an interface". It isn't necessary to know what this means to complete the project.
- We will cover interfaces next Monday. For project 1, just trust us!

Array Overview

Lecture 5, CS61B, Fall 2025

SLLists:

- Summary of SLLists So Far
- Why a Last Pointer Isn't Enough
- Doubly Linked Lists
- Generic Lists
- Linked Lists Are Bad at Get

ALists:

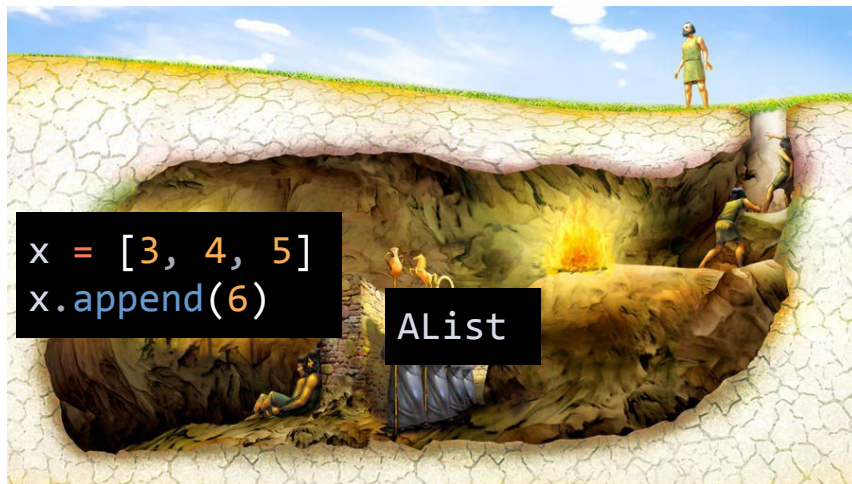
- **Array Review**
- Basic ArrayList Implementation
- removeLast and the Allegory of the Cave
- removeLast Implementation

Our Long Term Goal (next two lectures): The AList

In the last three lectures, we've seen how we can harness a recursive class definition to build an expandable list, ie. the `IntList`, the `SLList`, and the `DLList`.

In this lecture and in lecture 7 (Friday), we'll talk about a totally different way to implement lists that is based on an array.

- This is the approach used by Python (behind a layer of abstraction).



Getting Memory Boxes

To store information, we need memory boxes, which we can get in Java by declaring variables or instantiating objects. Examples:

- `int x;` ← Gives us a memory box of 32 bits that stores ints.
- `Walrus w1;` ← Gives us a memory box of 64 bits that stores Walrus references.
- `Walrus w2 = new Walrus(30, 5.6);` ← Gives us a memory box of 64 bits that stores Walrus references, and also gives us 96 bits for storing the int size (32 bits) and double tuskSize (64 bits) of our Walrus.

Arrays are a special kind of object which consists of a **numbered** sequence of memory boxes.

- To get ith item of array A, use `A[i]`.
- Unlike **class** instances which have have **named** memory boxes.

Arrays consist of:

- A fixed integer **length** (cannot change!)
- A sequence of N memory boxes where **N=length**, such that:
 - All of the boxes hold the same type of value (and have same # of bits).
 - The boxes are numbered 0 through length-1.

Like instances of classes:

- You get one reference when its created.
- If you reassign all variables containing that reference, you can never get the array back.

Unlike classes, arrays do not have methods.

Basic ArrayList Implementation

Lecture 5, CS61B, Fall 2025

SLLists:

- Summary of SLLists So Far
- Why a Last Pointer Isn't Enough
- Doubly Linked Lists
- Generic Lists
- Linked Lists Are Bad at Get

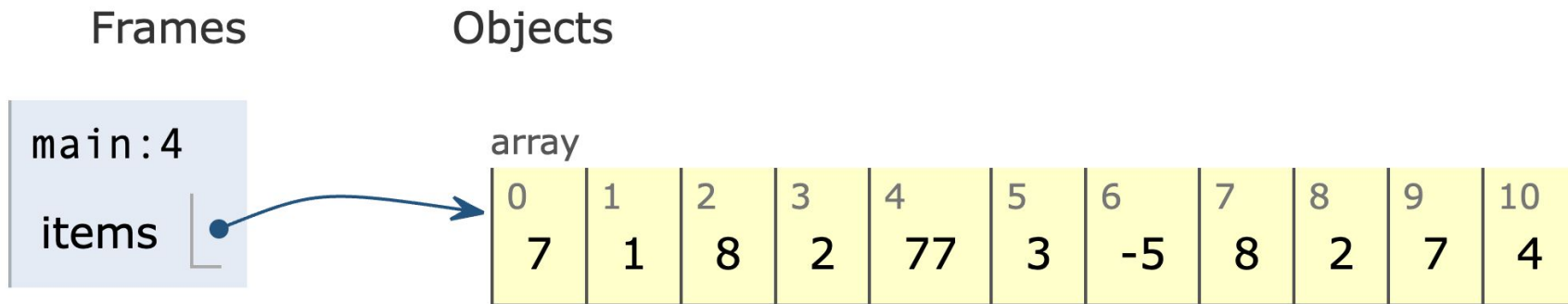
ALists:

- Array Review
- **Basic ArrayList Implementation**
- removeLast and the Allegory of the Cave
- removeLast Implementation

Random Access in Arrays

Retrieval from any position of an array is very fast.

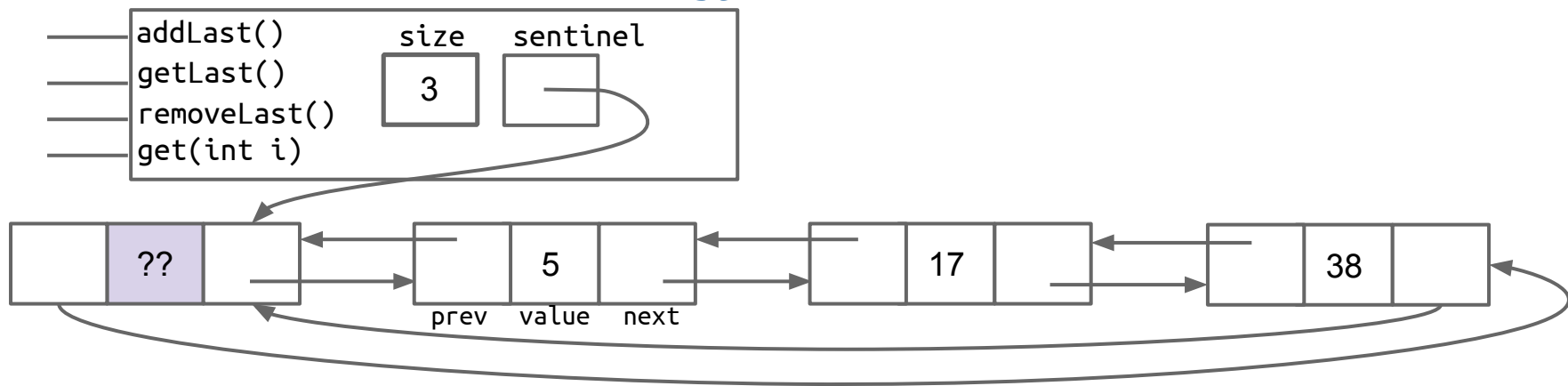
- Independent* of array size.
- 61C Preview: Ultra fast random access results from the fact that memory boxes are the same size (in bits).



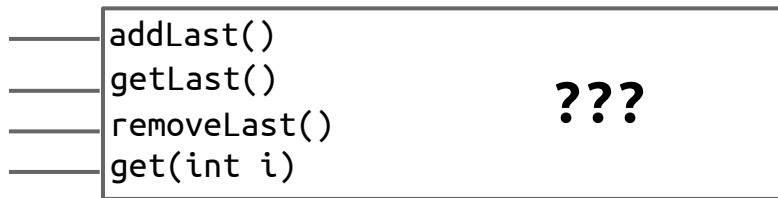
Want to figure out how to build an array version of a list:

- In lecture we'll only do back operations. Mini-project 2 will cover front operations.

DLList



AList



Let's try it out...

Coding Demo: Basic ArrayList Constructor

AList.java

```
public class AList {  
  
    /** Creates an empty list. */  
    public AList() {  
  
    }  
  
}
```

Coding Demo: Basic ArrayList Constructor

AList.java

```
public class AList {  
  
    private int size;  
  
    /** Creates an empty list. */  
    public AList() {  
  
    }  
  
}
```

Coding Demo: Basic ArrayList Constructor

AList.java

```
public class AList {  
    private int[] items;  
    private int size;  
  
    /** Creates an empty list. */  
    public AList() {  
  
    }  
  
}
```

Coding Demo: Basic ArrayList Constructor

AList.java

```
public class AList {  
    private int[] items;  
    private int size;  
  
    /** Creates an empty list. */  
    public AList() {  
        items = new int[100];  
    }  
}
```

The choice of array size (100) was arbitrary. We'll fix this limitation later.

Coding Demo: Basic ArrayList Constructor

AList.java

```
public class AList {  
    private int[] items;  
    private int size;  
  
    /** Creates an empty list. */  
    public AList() {  
        items = new int[100];  
        size = 0;  
    }  
}
```

Coding Demo: Basic ArrayList addLast

AList.java

```
public class AList {  
    private int[] items;  
    private int size;  
  
    /** Inserts x into the back of the list. */  
    public void addLast(int x) {  
  
    }  
  
}
```

Let's write a small example to help us think about addLast.

Coding Demo: Basic ArrayList addLast

0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---

0 1 2 3 4 5 6 7 8 ...

6	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---

0 1 2 3 4 5 6 7 8 ...

6	9	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---

0 1 2 3 4 5 6 7 8 ...

6	9	-1	0	0	0	0	0	0	0
---	---	----	---	---	---	---	---	---	---

0 1 2 3 4 5 6 7 8 ...

Call constructor to get empty array.

size=0

addLast(6)

size=1

addLast(9)

size=2

addLast(-1)

size=3

What patterns do we spot?

The next item we want to add will go into position size.

Coding Demo: Basic ArrayList addLast

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size

*/
public class AList {
    private int[] items;
    private int size;

    /** Inserts x into the back of the list. */
    public void addLast(int x) {

    }

}
```

Coding Demo: Basic ArrayList addLast

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size

*/
public class AList {
    private int[] items;
    private int size;

    /** Inserts x into the back of the list. */
    public void addLast(int x) {
        items[size] = x;
    }
}
```

Coding Demo: Basic ArrayList addLast

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size

*/
public class AList {
    private int[] items;
    private int size;

    /** Inserts x into the back of the list. */
    public void addLast(int x) {
        items[size] = x;
        size += 1;
    }
}
```

Coding Demo: Basic ArrayList getLast

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size

*/
public class AList {
    private int[] items;
    private int size;

    /** Returns the item from the back of the list. */
    public int getLast() {

    }

}
```

Coding Demo: Basic ArrayList getLast

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size
    getLast: The item we want to return is in position size - 1.

*/
public class AList {
    private int[] items;
    private int size;

    /** Returns the item from the back of the list. */
    public int getLast() {

    }

}
```


Coding Demo: Basic ArrayList getLast

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size
    getLast: The item we want to return is in position size - 1.

*/
public class AList {
    private int[] items;
    private int size;

    /** Returns the item from the back of the list. */
    public int getLast() {
        return items[size - 1];
    }
}
```

Coding Demo: Basic ArrayList get

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size
    getLast: The item we want to return is in position size - 1.

*/
public class AList {
    private int[] items;
    private int size;

    /** Gets the ith item in the list (0 is the front). */
    public int get(int i) {

    }

}
```

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size
    getLast: The item we want to return is in position size - 1.

*/
public class AList {
    private int[] items;
    private int size;

    /** Gets the ith item in the list (0 is the front). */
    public int get(int i) {
        return items[i];
    }
}
```

Coding Demo: Basic ArrayList size

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size
    getLast: The item we want to return is in position size - 1.

*/
public class AList {
    private int[] items;
    private int size;

    /** Returns the number of items in the list. */
    public int size() {

    }

}
```

Coding Demo: Basic ArrayList size

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size
    getLast: The item we want to return is in position size - 1.
    size: The number of items in the list should be size.
*/
public class AList {
    private int[] items;
    private int size;

    /** Returns the number of items in the list. */
    public int size() {

    }
}
```

Coding Demo: Basic ArrayList size

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size
    getLast: The item we want to return is in position size - 1.
    size: The number of items in the list should be size.
*/
public class AList {
    private int[] items;
    private int size;

    /** Returns the number of items in the list. */
    public int size() {
        return size;
    }
}
```

```
public class AList {
    private int[] items;
    private int size;

    public AList() {
        items = new int[100]; size = 0;
    }

    public void addLast(int x) {
        items[size] = x;
        size += 1;
    }

    public int getLast() {
        return items[size - 1];
    }

    public int get(int i) {
        return items[i];
    }

    public int size() {
        return size;
    }
}
```

AList Invariants:

- The position of the next item to be inserted is always size.
- size is always the number of items in the AList.
- The last item in the list is always in position size - 1.

We could also add error checking code, e.g.

```
public int get(int i) {
    if (i >= items.length) {
        throw new IllegalArgumentException();
    }
    return items[i];
}
```

```
public class AList {
    private int[] items;
    private int size;

    public AList() {
        items = new int[100]; size = 0;
    }

    public void addLast(int x) {
        items[size] = x;
        size += 1;
    }

    public int getLast() {
        return items[size - 1];
    }

    public int get(int i) {
        return items[i];
    }

    public int size() {
        return size;
    }
}
```

AList Invariants:

- The position of the next item to be inserted is always size.
- size is always the number of items in the AList.
- The last item in the list is always in position size - 1.

Let's now discuss delete operations.

removeLast and the Allegory of the Cave

Lecture 5, CS61B, Fall 2025

SLLists:

- Summary of SLLists So Far
- Why a Last Pointer Isn't Enough
- Doubly Linked Lists
- Generic Lists
- Linked Lists Are Bad at Get

ALists:

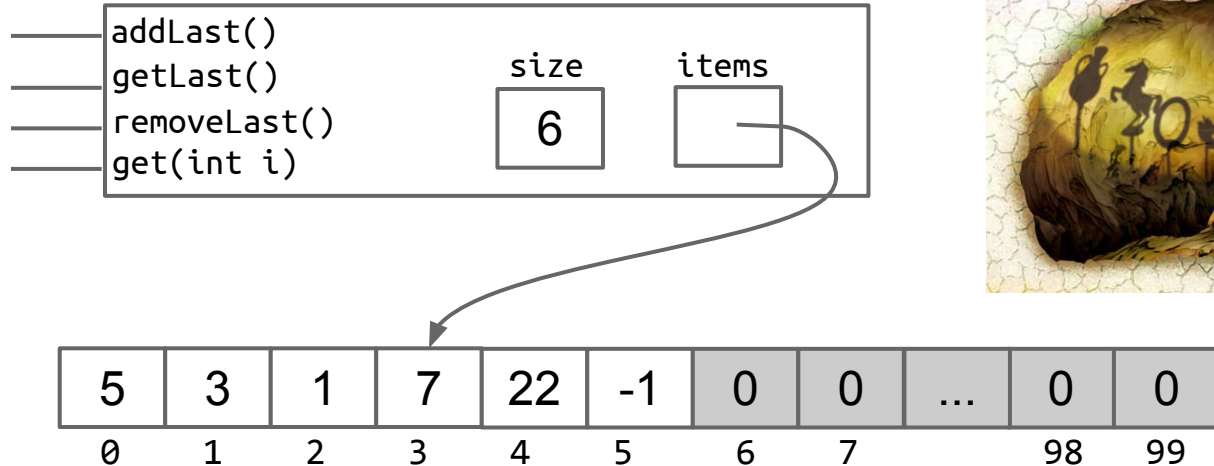
- Array Review
- Basic ArrayList Implementation
- **removeLast and the Allegory of the Cave**
- removeLast Implementation

The Abstract vs. the Concrete

When we `removeLast()`, which memory boxes need to change? To what?-

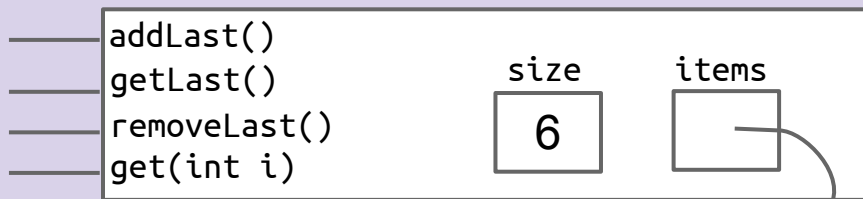
User's mental model: $[5, 3, 1, 7, 22, -1] \rightarrow [5, 3, 1, 7, 22]$

Actual truth:

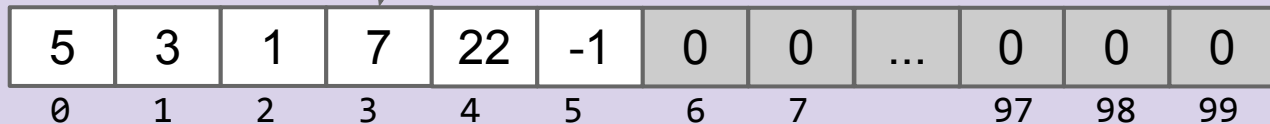




When we `removeLast()`, which memory boxes need to change? To what?



- a) `size`
- b) `size` and `items`
- c) `size` and `items[i]` for some `i`
- d) `size`, `items`, and `items[i]` for some `i`
- e) `size`, `items`, and `items[i]` for many different `i`



- The position of the next item to be inserted is always `size`.
- `size` is always the number of items in the AList.
- The last item in the list is always in position `size - 1`.

} AList invariants.

removeLast Implementation

Lecture 5, CS61B, Fall 2025

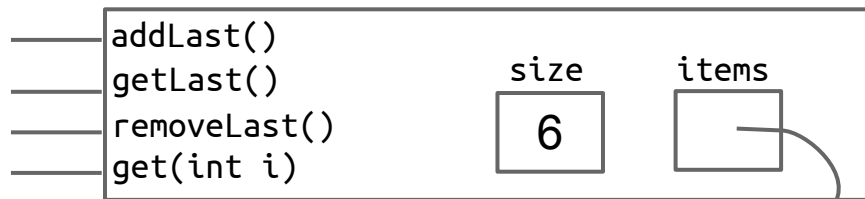
SLLists:

- Summary of SLLists So Far
- Why a Last Pointer Isn't Enough
- Doubly Linked Lists
- Generic Lists
- Linked Lists Are Bad at Get

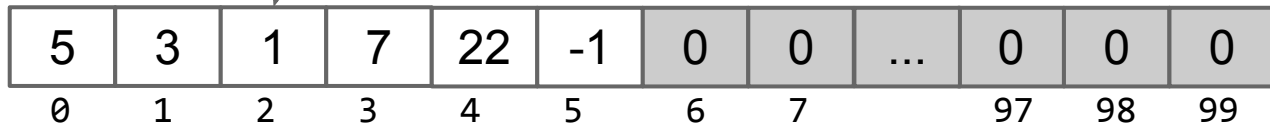
ALists:

- Array Review
- Basic ArrayList Implementation
- removeLast and the Allegory of the Cave
- **removeLast Implementation**

When we `removeLast()`, which memory boxes need to change? To what?



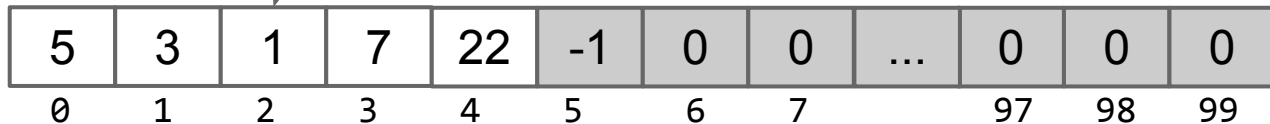
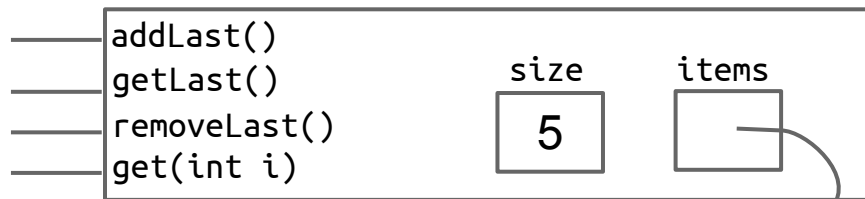
- a) **size**
- b) `size` and `items`
- c) `size` and `items[i]` for some `i`
- d) `size`, `items`, and `items[i]` for some `i`
- e) `size`, `items`, and `items[i]` for many different `i`



- The position of the next item to be inserted is always `size`.
- `size` is always the number of items in the AList.
- The last item in the list is always in position `size - 1`.

} AList invariants.

When we `removeLast()`, which memory boxes need to change? To what?



- a) **size**
- b) size and items
- c) size and items[i] for some i
- d) size, items, and items[i] for some i
- e) size, items, and items[i] for many different i

- The position of the next item to be inserted is always `size`.
- `size` is always the number of items in the AList.
- The last item in the list is always in position `size - 1`.

} AList invariants.

Coding Demo: Basic ArrayList removeLast

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size
    getLast: The item we want to return is in position size - 1.
    size: The number of items in the list should be size.
*/
public class AList {
    private int[] items;
    private int size;

    /** Deletes item from back of list and returns deleted item. */
    public int removeLast() {

    }
}
```

Coding Demo: Basic ArrayList removeLast

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size
    getLast: The item we want to return is in position size - 1.
    size: The number of items in the list should be size.
*/
public class AList {
    private int[] items;
    private int size;

    /** Deletes item from back of list and returns deleted item. */
    public int removeLast() {
        int x = getLast();

    }
}
```


Coding Demo: Basic ArrayList removeLast

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size
    getLast: The item we want to return is in position size - 1.
    size: The number of items in the list should be size.
*/
public class AList {
    private int[] items;
    private int size;

    /** Deletes item from back of list and returns deleted item. */
    public int removeLast() {
        int x = getLast();

        return x;
    }
}
```

Coding Demo: Basic ArrayList removeLast

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size
    getLast: The item we want to return is in position size - 1.
    size: The number of items in the list should be size.
*/
public class AList {
    private int[] items;
    private int size;

    /** Deletes item from back of list and returns deleted item. */
    public int removeLast() {
        int x = getLast();
        size = size - 1;
        return x;
    }
}
```

Naive AList Code

```
public class AList {
    private int[] items;
    private int size;

    public AList() {
        items = new int[100]; size = 0;
    }

    public void addLast(int x) {
        items[size] = x;
        size += 1;
    }

    public int getLast() {
        return items[size - 1];
    }

    public int get(int i) {
        return items[i];
    }

    public int size() {
        return size;
    }
}
```

AList Invariants:

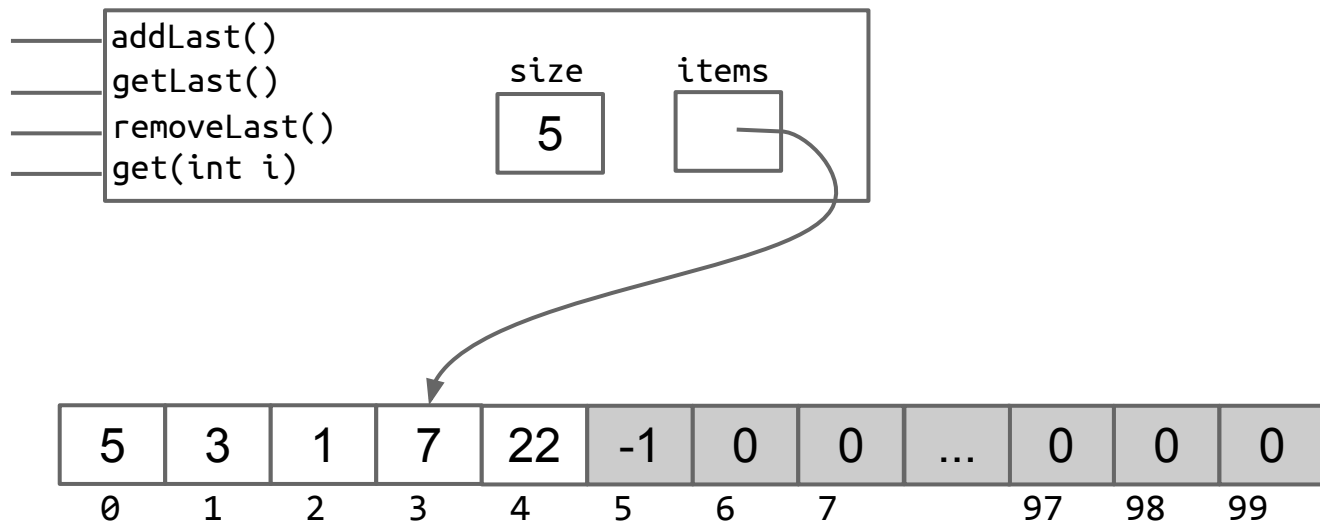
- The position of the next item to be inserted is always size.
- size is always the number of items in the AList.
- The last item in the list is always in position size - 1.

```
public int removeLast() {
    int x = items[size - 1];
    items[size - 1] = 0;
    size -= 1;
    return x;
}
```

Setting deleted item to zero is not necessary to preserve invariants, and thus not necessary for correctness.

What about get?

- Some students suggest we should set the value to zero so that we can't get(5).
- There's no specified behavior for what to do when get is out of bounds.
 - IMO an exception is best as shown earlier.



```
public class AList {  
    private int[] items;  
    private int size;  
  
    public AList() {  
        items = new int[100]; size = 0;  
    }  
  
    public void addLast(int x) {  
        items[size] = x;  
        size += 1;  
    }  
  
    public int getLast() {  
        return items[size - 1];  
    }  
  
    public int get(int i) {  
        return items[i];  
    }  
  
    public int size() {  
        return size;  
    }  
}
```

What's missing from our implementation?

```
public int removeLast() {  
    int x = items[size - 1];  
    items[size - 1] = 0;  
    size -= 1;  
    return x;  
}
```

Naive AList Code

```
public class AList {
    private int[] items;
    private int size;

    public AList() {
        items = new int[100]; size = 0;
    }

    public void addLast(int x) {
        items[size] = x;
        size += 1;
    }

    public int getLast() {
        return items[size - 1];
    }

    public int get(int i) {
        return items[i];
    }

    public int size() {
        return size;
    }
}
```

What's missing from our implementation?

```
public int removeLast() {
    int x = items[size - 1];
    items[size - 1] = 0;
    size -= 1;
    return x;
}
```

```
public class AList {
    private int[] items;
    private int size;

    public AList() {
        items = new int[100]; size = 0;
    }

    public void addLast(int x) {
        items[size] = x;
        size += 1;
    }

    public int getLast() {
        return items[size - 1];
    }

    public int get(int i) {
        return items[i];
    }

    public int size() {
        return size;
    }
}
```

In Lecture 7 (Friday), we'll cover:

- “Expanding” our array to support more than 100 items when necessary.
- Making our AList generic.

However, in lecture 6, we'll take a detour to cover the idea of software testing (which is also covered on mini-project 1).

```
public int removeLast() {
    int x = items[size - 1];
    items[size - 1] = 0;
    size -= 1;
    return x;
}
```

Attendance link: <https://www.yellkey.com/offline>

To-do: If you haven't finished it, complete the Stacks homework.

- Mini-project 1 is released, due next Monday.
- Debugging tools HW due Friday.

In Stacks:

- Use SLList as a guide, but don't literally use an SLList.



Bonus Topic: Arrays vs. Classes

Lecture 5, CS61B, Spring 2025

Arrays vs. Classes

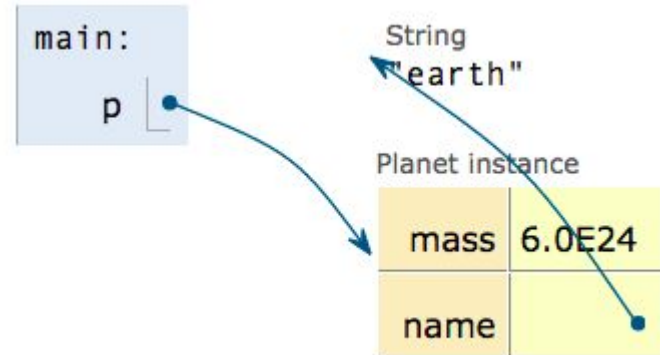
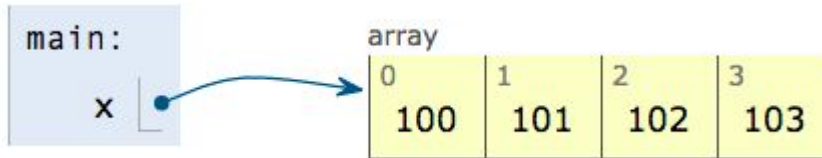
Arrays vs. Classes

Arrays and Classes can both be used to organize a bunch of memory boxes.

- Array boxes are accessed using `[]` notation.
- Class boxes are accessed using dot notation.
- Array boxes must all be of the same type.
- Class boxes may be of different types.
- Both have a fixed number of boxes.

```
public class Planet {  
    public double mass;  
    public String name;  
    ...  
}
```

```
int[] x = new int[]{100, 101, 102, 103};  
Planet p = new Planet(6e24, "earth");
```

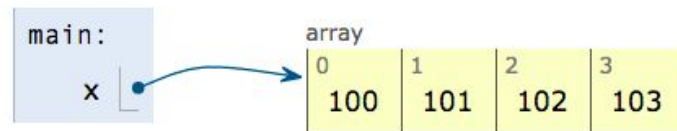


Arrays vs. Classes

Array indices can be computed at runtime.

```
int[] x = new int[]{100, 101, 102, 103};  
int indexOfInterest = askUser();  
int k = x[indexOfInterest];  
System.out.println(k);
```

```
jug ~/Dropbox/61b/lec/lists3  
$ javac ArrayDemo.java  
$ java ArrayDemo  
What index do you want? 2  
102
```

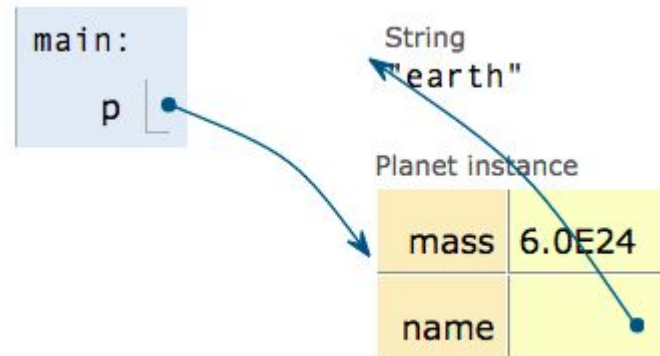


Arrays vs. Classes

Class member variable names CANNOT be computed and used at runtime.

```
String fieldOfInterest = "mass";  
Planet earth = new Planet(6e24, "earth");  
double mass = earth[fieldOfInterest];  
System.out.println(mass);
```

```
jug ~/Dropbox/61b/lec/lists3  
$ javac ClassDemo.java  
ClassDemo.java:5: error: array required,  
    but Planet found.  
  
    double mass = earth[fieldOfInterest];  
                        ^
```



... if you reallllly want to do this, you can: <https://goo.gl/JxpyLq>

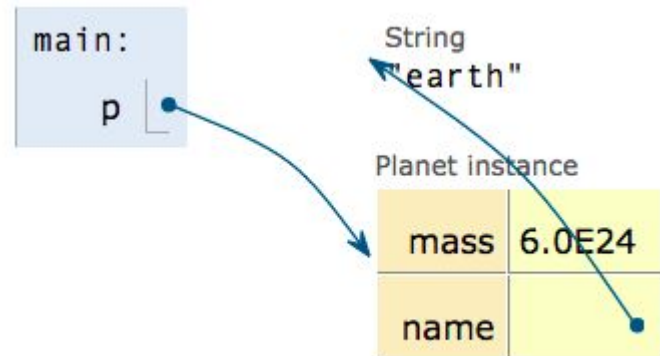
Arrays vs. Classes

Class member variable names CANNOT be computed and used at runtime.

- Dot notation doesn't work either.

```
String fieldOfInterest = "mass";  
Planet earth = new Planet(6e24, "earth");  
double mass = earth.fieldOfInterest;  
System.out.println(mass);
```

```
jug ~/Dropbox/61b/lec/lists3  
$ javac ClassDemo.java  
ClassDemo.java:5: error: cannot find Symbol  
    double mass = earth.fieldOfInterest;  
                        ^  
symbol:   variable fieldOfInterest  
location: variable earth of type Planet
```



... if you reallllly want to do this, you can: <https://goo.gl/JxpyLq>

The only (easy) way to access a member of a class is with hard-coded dot notation.

```
int k = x[indexOfInterest];      /* no problem */  
  
double m = p.fieldOfInterest;    /* won't work */  
double z = p[fieldOfInterest];  /* won't work */  
/* No (sane) way to use field of interest */  
  
double w = p.mass;              /* works fine */
```

The Java compiler does not treat text on either side of a dot as an expression, and thus it is not evaluated.

- See a compilers or programming languages class for more!

*: There is something called the “reflections” library which will let you access a field of a class using a String name, but it is not intended for casual use.